



AI-Driven Lab Exam Monitoring and Management System

Final Year Project Proposal
by

Muhammad Haider Ali (2280354)

Zain Ali Mehmood (2280133)

Roshaan Ali Shah (2280327)

Supervised by: Ms. Zainab Iftikhar

Faculty of Computing and Engineering Sciences
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
Islamabad, Pakistan

Spring 2026

Project Description

The AI-Driven Lab Exam Monitoring and Management System is a secure examination platform designed for university computer laboratories. The system provides controlled exam execution, real-time monitoring, AI-assisted evaluation, and plagiarism detection within a unified framework.

It enforces authorized device access, restricts unauthorized applications through a lockdown mechanism, collects monitoring evidence during exams, and automates the grading for MCQ, coding, and theory-based assessments.

The proposed solution aims to improve fairness, efficiency, and academic integrity in laboratory-based examinations.

1 Introduction

With the increasing digitalization of higher education, universities are conducting a growing number of examinations in computer laboratories. These exams often involve programming, database management, and other technical tasks that require controlled system access and strict supervision.

Despite being conducted in a controlled environment, lab-based examinations face several challenges. Students may attempt to access unauthorized applications, switch tasks, use external storage devices, or collaborate unfairly. Manual invigilation alone is often insufficient to monitor all students effectively, especially in large classes. Additionally, grading technical exams manually is time-consuming and may lead to inconsistencies. Detecting plagiarism within the same class is also difficult without automated analysis tools.

These limitations highlight the need for a secure, intelligent, and integrated system tailored specifically for laboratory examination environments.

This proposal presents the design and development of an AI-Driven Lab Exam Monitoring and Management System that aims to enhance examination security, automate evaluation, and promote academic integrity in university lab settings.

2 Application/Literature Review

Several examination and proctoring systems have been developed to manage digital exams and reduce cheating. However, most existing solutions focus either on online quiz delivery, browser-level lockdown, or remote proctoring. They typically do not provide a unified, lab-focused system that combines operating system restrictions, evidence collection, AI-assisted evaluation, and integrated reporting.

Moodle Quiz [1] is a widely used Learning Management System (LMS) feature that supports online quizzes and assessments. It provides tools for quiz scheduling, question banks, grading for objective questions, and reports.

Features:

- Quiz creation and scheduling
- Question bank management
- Automatic grading for MCQs
- Time limits and attempt control
- Basic reports and statistics

Limitations:

- No operating-system level lockdown for lab machines
- Limited real-time monitoring during the exam
- Does not natively restrict desktop applications (e.g., compiler-only access)
- Limited support for automated evaluation of coding problems

Safe Exam Browser (SEB) [2] is a secure browser environment used for e-assessments. It turns a computer into a restricted workstation by controlling access to system functions, websites, and applications during an exam.

Features:

- Browser-based lockdown
- Restricts access to websites and system functions
- Can restrict applications based on configuration
- Used with LMS systems such as Moodle

Limitations:

- Monitoring evidence (screenshots/webcam) is not a complete built-in solution
- No AI-assisted evaluation for coding/theory exams
- No integrated plagiarism detection and reporting

Respondus LockDown Browser [3] is another exam security tool that locks down the testing environment within an LMS. When enabled, students are prevented from printing, copying, accessing other applications, or navigating away until the exam is submitted.

Features:

- Locks down the exam environment within supported LMS platforms
- Prevents switching to other applications during the exam
- Restricts printing, copying, and other actions
- Optional webcam-based monitoring through companion tools

Limitations:

- Primarily designed for browser-based exams, not full lab workflows
- Limited integration of AI grading for coding/theory exams
- Plagiarism reporting is not the core focus and may require external tools

ProctorU [4] is an online proctoring service designed mainly for remote examinations. It supports identity verification, secure browser usage, and live/recorded monitoring to reduce cheating.

Features:

- Live and/or recorded remote proctoring
- Identity verification and session monitoring
- Incident reporting and exam-day guidance

Limitations:

- Focused on remote exams rather than controlled lab environments
- Cost and privacy concerns for some institutions
- Limited operating-system level control in a lab-specific setup

Based on the reviewed systems, there is a gap for a laboratory-focused solution that combines strict lockdown control, continuous evidence collection, AI-assisted evaluation, and integrated plagiarism detection within a single platform.

3 Problem Statement

Universities increasingly conduct technical examinations in computer laboratories for programming, database, and software-related courses [9]. Although these exams are held in controlled environments, maintaining strict academic integrity remains a significant challenge.

During lab-based examinations, students may attempt to switch applications, access unauthorized software, use external storage devices, or collaborate with peers [8]. Manual invigilation alone may not effectively detect subtle violations, especially in large classes

Table 1: Applications Comparison

<i>Features</i>	Applications			
	Moodle [1]	SEB [2]	Respondus [3]	Proposed System
Exam Scheduling & Delivery	✓	✓	✓	✓
OS/App Restriction (Lockdown)	✗	✓	✓	✓
Real-time Evidence Collection	✗	✗	✓	✓
AI-assisted Evaluation	✗	✗	✗	✓
Intra-class Plagiarism Detection	✗	✗	✗	✓
Lab-based Device Control	✗	✗	✗	✓

where a single invigilator supervises dozens of students simultaneously. This increases the risk of unfair practices and reduces trust in the examination process.

Furthermore, grading coding and theory-based exams manually is time-consuming and prone to inconsistencies. In large classes, evaluating each submission individually requires significant effort from instructors. Additionally, detecting plagiarism among students within the same class is difficult without automated similarity analysis tools. Automated plagiarism detection systems such as MOSS [5] have been widely adopted in programming courses to identify similarity between student submissions. Comparative studies of source-code plagiarism detection engines [6, 7] highlight the importance of structured similarity analysis mechanisms in maintaining academic integrity.

Existing solutions such as LMS-based quiz systems, browser-level lockdown tools, or remote proctoring services address only partial aspects of the problem. They often lack full operating system-level control in laboratory environments, integrated AI-assisted evaluation for coding and theory exams, and comprehensive reporting with audit trails.

The unmet need is therefore a unified, laboratory-focused examination management system that enforces strict system control, provides real-time monitoring, automates evaluation, and detects plagiarism within a single integrated platform.

The primary stakeholders for such a system include universities, examination departments, faculty members, and academic institutions conducting technical lab-based assessments. As higher education institutions continue to digitize examination processes, the demand for secure and intelligent lab examination systems is expected to grow significantly.

4 Project Aim and Objectives

Project Aim:

The primary aim of this project is to design and implement a secure, intelligent, and integrated laboratory examination management system that ensures controlled exam execution, real-time monitoring, automated evaluation, and academic integrity enforcement within university computer labs.

Value Proposition:

Unlike existing solutions that focus either on online quiz management, browser-level restriction, or remote proctoring, the proposed system provides a unified laboratory-focused architecture. It combines operating system-level lockdown, continuous monitoring, AI-assisted grading, and intra-class plagiarism detection into a single integrated platform specifically designed for controlled lab environments.

Project Objectives:

- Develop a secure authentication system with authorized laboratory device access control.
- Implement a lockdown-based exam execution environment with strict application restrictions.
- Design a monitoring system that captures screenshots, webcam evidence, and system activity during exams.
- Develop AI-assisted evaluation for MCQ, coding, and theory-based assessments.
- Implement intra-class plagiarism detection using similarity analysis techniques.
- Provide a teacher dashboard for live proctoring and administrative control.
- Generate structured reports, audit logs, and analytics for transparency and review.
- Ensure system scalability, reliability, and security for multiple lab environments.

Final Project Output:

The final deliverable of this project will include:

- A desktop-based student examination application (Windows-based lab environment).
- A web-based teacher and administrator control panel.

- A backend API server for data processing and communication.
- A centralized database for exam data, monitoring records, and logs.
- An AI processing module for automated grading and similarity detection.

The system will be deployed on a university-controlled server environment and tested within a laboratory setup to simulate real examination conditions.

5 Scope and Significance

5.1 Project Scope

The scope of the proposed system is limited to laboratory-based examinations conducted within university-controlled computer labs. The system focuses on secure exam execution, monitoring, automated evaluation, and structured reporting within a controlled environment.

The project includes the development of the following core modules:

1. User Authentication and Role Management
2. Device Authorization and Access Control
3. Exam Creation and Configuration
4. Exam Scheduling and Dashboard Interface
5. Pre-Exam System Verification and Mock Testing
6. Secure Lockdown Exam Environment
7. Monitoring and Evidence Collection
8. Behavior Analysis and Violation Detection
9. Live Proctoring Dashboard
10. Submission and Auto-Save Mechanism
11. AI-Assisted Evaluation and Plagiarism Detection
12. Reporting and Audit Trail System

The system is designed specifically for institutional lab settings. Remote home-based examinations, biometric hardware integration, and cross-university cloud deployment are beyond the scope of this project.

5.2 Project Significance

The proposed system enhances the integrity and transparency of laboratory examinations by providing structured control, monitoring, and evaluation mechanisms within a single unified platform.

Key benefits of the system include:

- Improved academic integrity through strict system-level restrictions.
- Reduced instructor workload through automated grading.
- Faster and more consistent evaluation of coding and theory exams.
- Automated plagiarism detection within the same class.
- Centralized monitoring and audit logging for accountability.
- Scalability for multiple lab sections and large student groups.

By integrating security, monitoring, evaluation, and reporting into one system, the project contributes toward a more reliable and efficient digital examination framework for higher education institutions.

6 Project Development Methodology

The proposed system will be developed using an iterative and modular development approach. The project will be divided into structured phases including analysis, design, implementation, testing, and integration. Each core module will be developed and tested independently before system-level integration.

Development Approach

An iterative development model will be adopted to ensure continuous improvement and early detection of issues. Each module will be implemented as a separate component and validated before integration with other modules. This approach ensures flexibility, better debugging, and progressive system refinement.

System-Level Architecture Overview

The system will follow a client-server architecture consisting of the following major components:

- Student Examination Application (Desktop Client)
- Teacher/Admin Web Panel

- Backend API Server
- Centralized Database Server
- AI Evaluation and Analysis Module

The student application will run in a controlled laboratory environment and communicate securely with the backend server. Monitoring data and exam submissions will be transmitted to the server for storage and analysis. The teacher panel will access processed data for real-time monitoring and post-exam evaluation.

Module-wise Development and Deliverables

Phase 1: Core Access Control Modules

- Authentication and Role Management
- Device Authorization
- Eligibility Verification

Deliverable: Secure login and access control prototype.

Phase 2: Exam Execution Modules

- Exam Creation and Scheduling
- Lockdown Environment Implementation
- Pre-Exam Verification

Deliverable: Functional exam execution environment with restrictions.

Phase 3: Monitoring and Detection Modules

- Screenshot and Webcam Capture
- Activity Logging
- Suspicious Behavior Detection
- Live Proctoring Dashboard

Deliverable: Real-time monitoring and alert system.

Phase 4: Evaluation and Reporting Modules

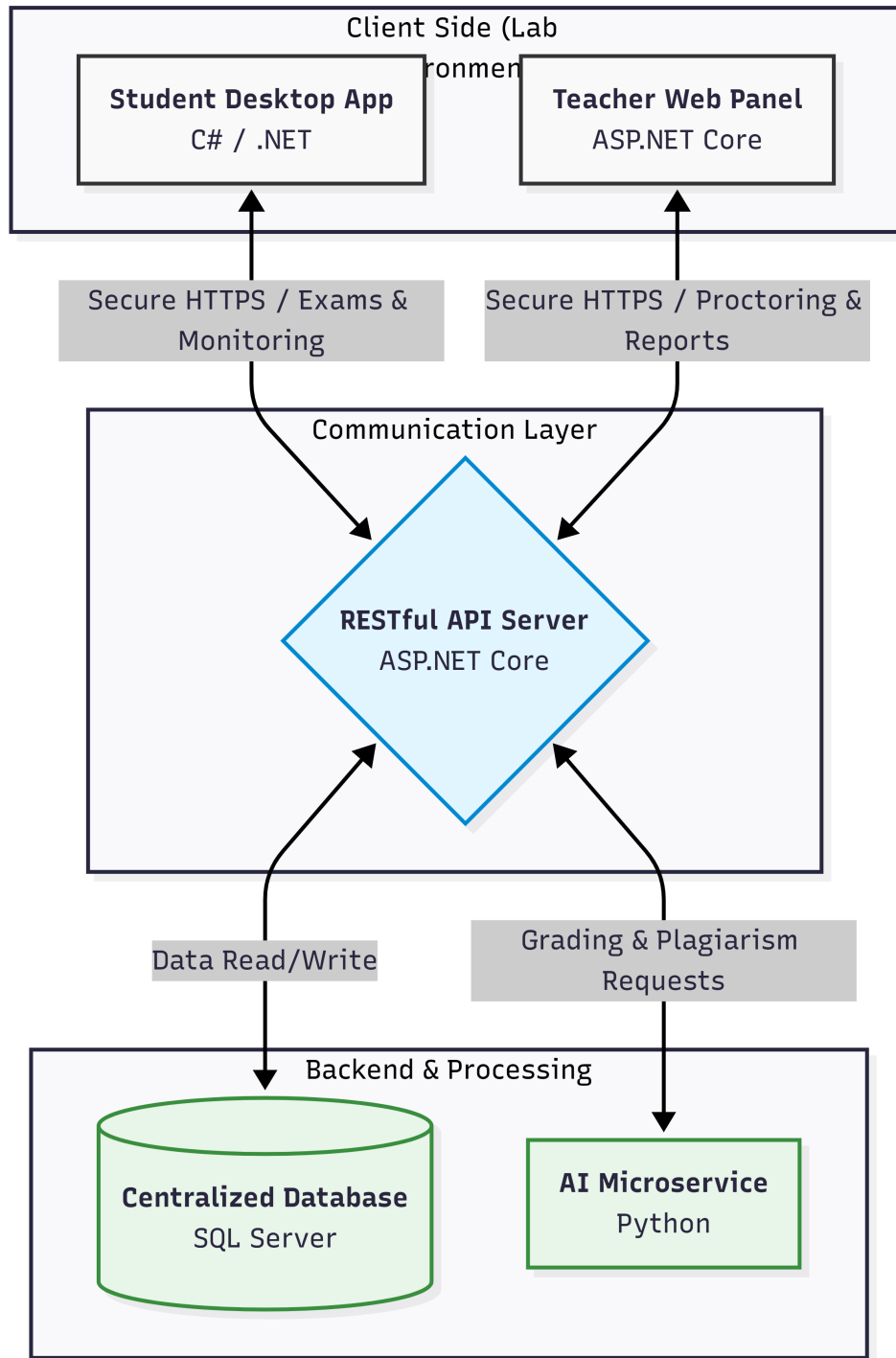


Figure 1: System Architecture Block Diagram

- AI-Assisted MCQ Grading
- Coding Test Case Evaluation
- Theory Similarity Analysis
- Plagiarism Detection
- Reporting and Audit Logs

Deliverable: Automated grading system and structured reports.

7 Tools and Technologies

The implementation of the AI-Driven Lab Exam Monitoring and Management System will involve a combination of desktop application development, web technologies, backend services, and AI-based processing tools.

1. Student Examination Application: The student-side examination client will be developed using C# with .NET (WPF or WinForms). This platform is chosen because it provides strong integration with Windows-based laboratory systems and allows operating system-level control required for implementing lockdown mechanisms and application restrictions.

2. Backend API Server: ASP.NET Core will be used to develop a RESTful API for secure communication between the student application, teacher panel, and database server. It provides scalability, performance, and built-in security features such as authentication and role-based access control.

3. Teacher and Admin Web Panel: The web-based dashboard will be developed using ASP.NET Core with Razor Pages or a modern frontend framework (such as React, if required). This interface will allow teachers to create exams, monitor students, and generate reports.

4. Database Management System: SQL Server or PostgreSQL will be used as the relational database system to store user data, exam configurations, monitoring records, submissions, and logs. A relational database is suitable due to the structured nature of examination data.

5. AI and Similarity Analysis Module: Python may be integrated as a microservice for implementing similarity analysis and automated grading algorithms. Modern AI-based evaluation approaches such as Deep Knowledge Tracing [10] demonstrate how machine learning can model student performance patterns. Similarly, clustering-based grading techniques [11] show that automated short-answer and structured-response grading can reduce instructor workload while maintaining consistency.

6. Communication and Security: Secure HTTP (HTTPS) protocols will be used for data transmission between clients and servers. Authentication tokens and encrypted session handling will ensure secure access control.

7. Development and Version Control Tools: Visual Studio will be used as the primary development environment. Git will be used for version control and collaborative development among team members.

This combination of technologies ensures secure system-level control, scalable backend processing, structured data management, and reliable deployment within a university-controlled laboratory environment.

8 Work Plan

8.1 Team Structure

The successful execution of this project relies on a structured division of roles. Table 2 outlines the core team members and their primary responsibilities.

Table 2: Team Structure

Sr. No.	Team Members	Role
1	Muhammad Haider Ali	Project Manager & Backend Developer
2	Zain Ali Mehmood	Desktop Application Developer
3	Roshaan Ali Shah	AI Module Developer & QA Engineer

8.2 Work Distribution

To ensure efficient progress, development tasks have been divided among the team based on individual expertise. Table 3 details the specific work assignments for the project.

Table 3: Work Distribution

Sr. No.	Team Member	Work Assignment
1	Muhammad Haider Ali	System Architecture Design, Backend API Development, Database Design
2	Zain Ali Mehmood	Student Desktop Application, Lockdown Implementation, Monitoring Client
3	Roshaan Ali Shah	AI Evaluation Module, Plagiarism Detection, Testing & Validation

8.3 Gantt Chart

Fig. 2 presents the planned development timeline of the project. The work is divided into structured phases including requirement analysis, system design, core module im-

plementation, monitoring and AI module development, system integration, testing, and final deployment.

Major milestones include:

- Completion of system architecture design
- Implementation of secure exam execution modules
- Integration of monitoring and AI evaluation modules
- Full system testing and validation
- Final deployment and documentation

The Gantt chart ensures structured progress tracking and timely completion of project deliverables.

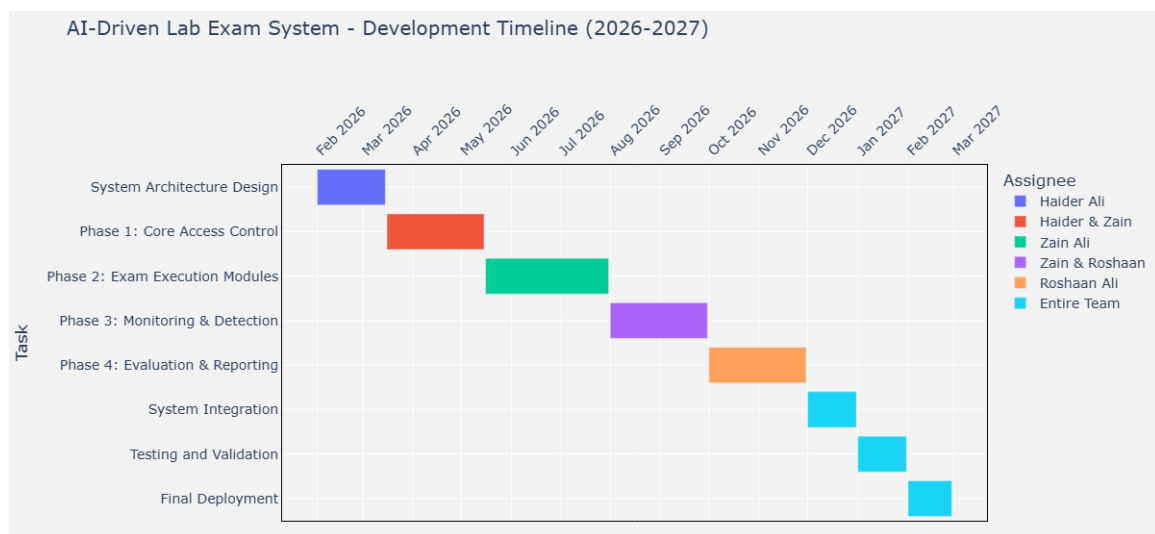


Figure 2: Gantt Chart

References

- [1] Moodle. “Quiz Activity Documentation.” Available: https://docs.moodle.org/en/Quiz_activity Accessed: February 2026.
- [2] Safe Exam Browser. “About Safe Exam Browser.” Available: <https://safeexambrowser.org/> Accessed: February 2026.
- [3] Respondus. “LockDown Browser Overview.” Available: <https://web.respondus.com/he/lockdownbrowser/> Accessed: February 2026.
- [4] ProctorU. “Online Proctoring Platform.” Available: <https://www.proctoru.com/> Accessed: February 2026.
- [5] Aiken, A. (1994). “MOSS: A System for Detecting Software Plagiarism.” University of California, Berkeley.
- [6] Lancaster, T., & Culwin, F. (2004). “A Comparison of Source Code Plagiarism Detection Engines.” *Computer Science Education*, 14(2), 101–112.
- [7] Clough, P. (2000). “Plagiarism in Natural and Programming Languages: An Overview of Current Tools and Technologies.” Department of Computer Science, University of Sheffield.
- [8] Dawson, P. (2016). “Five Ways to Hack and Cheat with Bring-Your-Own-Device Electronic Examinations.” *British Journal of Educational Technology*, 47(4), 592–600.
- [9] Kumar, A. N. (2005). “The Effect of Closed Labs in Computer Science Courses.” *SIGCSE Bulletin*, 37(3), 9–13.
- [10] Piech, C., et al. (2015). “Deep Knowledge Tracing.” *Advances in Neural Information Processing Systems (NeurIPS)*.
- [11] Basu, S., Jacobs, C., & Vanderwende, L. (2013). “Powergrading: A Clustering Approach to Amplify Human Effort for Short Answer Grading.” *Transactions of the Association for Computational Linguistics*, 1, 391–402.